

AGEX Agent Framework

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX에서 Agent를 공식 Resource이자 Runtime Execution Actor로 정의하고, Agent의 구조, Lifecycle, Context, Model 사용, Knowledge/Memory 접근, Tool 사용, Planning, Action, Delegation, Approval, Security, Governance, Observability, Evaluation 및 Acceptance 기준을 규정한다.

AGEX에서 Agent는 단순 Prompt Template이나 LLM 호출 Wrapper가 아니다.

Agent는 다음 능력을 가진 실행 주체다.

- 목표 이해
- Context 구성
- Knowledge 검색
- Memory 회상
- 계획 수립
- Action 선택
- Tool 사용
- 다른 Agent로 Delegation
- Workflow 호출
- 결과 검증
- 상태 갱신
- 완료 또는 실패 판단

그러나 모든 행동은 다음 경계 안에서 이루어진다.

Tenant

∩

Identity

∩

Permission

∩

Policy

∩

Data Classification

∩

Runtime Constraint

∩

Budget

∩

Governance

본 문서는 AGEX Runtime Architecture, Workflow Engine, Knowledge Engine, Memory Engine, Plugin Framework, Model Gateway, Security Architecture 및 Governance의 상위 계약과 일관되어야 한다.

2. Agent Framework의 공식 정의

AGEX Agent는 명시된 목표와 역할을 기준으로 Context를 이해하고, Knowledge·Memory·Model·Tool 및 다른 Agent를 활용하여 계획과 Action을 선택하되, 모든 행동이 Tenant·Permission·Policy·Runtime·Budget·Governance 통제 아래에서 수행되는 Versioned AI Execution Resource이다.

Agent Framework는 이러한 Agent를 생성·검증·Publish·실행·관찰·평가·통제하기 위한 AGEX의 공식 Framework이다.

3. Agent Framework의 목표

AGEX Agent Framework는 다음을 달성해야 한다.

1. Agent를 재사용 가능한 공식 Resource로 관리한다.
2. Prompt와 Runtime Permission을 분리한다.

3. 특정 Model Provider와 Agent를 분리한다.
 4. Agent 실행을 Durable Runtime 위에서 수행한다.
 5. Knowledge와 Memory를 명시적 Scope로 연결한다.
 6. Tool 사용을 Permission과 Policy로 통제한다.
 7. Agent의 자율성을 단계적으로 제한한다.
 8. Multi-Agent Delegation을 구조화한다.
 9. Agent 실행을 재현 가능하게 한다.
 10. 운영자가 행동 근거와 결과를 추적할 수 있게 한다.
 11. Cost와 Resource 사용을 실행 단위로 측정한다.
 12. AI 품질을 Evaluation으로 검증한다.
 13. Agent가 자신의 권한을 확대할 수 없도록 한다.
 14. 특정 산업이나 Application 업무를 Agent Framework에 내장하지 않는다.
-

4. 설계 원칙

AGEX Agent Framework는 다음 원칙을 따른다.

- Agent as First-Class Resource
- Runtime-Centered Execution
- Goal-Oriented
- Model Agnostic
- Tool Agnostic
- Knowledge Native
- Memory Native
- Workflow Native
- Explicit Capability
- Explicit Permission

- Bounded Autonomy
 - Policy Before Action
 - Version-Pinned Execution
 - Structured Context
 - Structured Action
 - Structured Result
 - Human Governed Autonomy
 - Least Privilege
 - Fail-Safe Default
 - Tenant Isolation
 - Observable by Default
 - Evaluation Before Promotion
 - No Self-Privilege Escalation
 - No Hidden State
 - No Raw Chain-of-Thought Dependency
-

5. Agent와 일반 LLM 호출의 차이

일반 LLM 호출:

Input

→ Model

→ Output

AGEX Agent:

Goal

→ Context

→ Evaluate

- Plan
- Select Action
- Validate
- Execute
- Observe
- Update State
- Continue / Complete

Agent는 단순 응답 생성이 아니라 실행 상태와 Action Lifecycle을 가진다.

6. Agent와 Workflow의 차이

Agent와 Workflow는 서로 대체 관계가 아니다.

Agent

확률적 판단 주체.

- 이해
- 계획
- Action 선택
- 추론
- Delegation

Workflow

명시적 실행 제어 구조.

- 순서
- 분기
- Wait
- Retry
- Approval

- Compensation

개념적으로:

Workflow = Execution Control

Agent = Decision Actor

AGEX는 둘을 조합한다.

7. Agent Resource 구조

Agent Resource는 최소 다음 영역으로 구성한다.

Agent

├ Identity

├ Metadata

├ Role

├ Goal

├ Instruction

├ Capability

├ Permission

├ Autonomy

├ Model Policy

├ Context Policy

├ Knowledge Binding

├ Memory Policy

├ Tool Binding

├ Planning Policy

├ Collaboration Policy

├ Execution Policy

└─ Evaluation Policy

└─ Lifecycle

8. Agent Specification과 Status

Agent는 Specification과 Status를 분리한다.

8.1 Specification

사용자가 정의하는 Desired Agent.

예:

- Goal
- Instruction
- Capability
- Permission
- Model Policy
- Knowledge
- Memory
- Tools
- Autonomy

8.2 Status

시스템이 관리하는 현재 상태.

예:

- Lifecycle State
- Health
- Active Version
- Last Evaluation
- Last Execution

- Suspension Reason

Runtime이 Status를 수정하며 Application이 직접 조작하지 않는다.

9. Agent 기본 Resource 예

개념 예:

id: agt_xxx

type: agent

tenant_id: ten_xxx

name: example-agent

version: 3

revision: 1

specification:

role: assistant

goal: >

주어진 요청을 분석하고 허용된 정보와 도구를 이용하여
목표를 달성한다.

autonomy_level: L2

model_policy:

profile_id: mdlprof_xxx

knowledge:

collections:

- knc_xxx

memory:

read:

- session
- agent

write_mode: proposal

tools:

- tool.item.lookup

status:

state: active

실제 Schema는 공식 Canonical Schema를 따른다.

10. Agent Identity

Agent는 고유 Identity를 가진다.

구분:

- Agent Definition Identity
- Agent Version Identity
- Execution Identity

Agent Definition과 실제 실행 Actor를 동일하게 취급하지 않는다.

예:

Agent Definition

agt_123

Agent Version

agt_123@v4

Execution Identity

exe_789

11. Agent Role

Role은 Agent의 기능적 성격을 표현한다.

예:

- Coordinator
- Planner
- Researcher
- Validator
- Executor

Role은 설명적 개념이며 Security Permission을 대신하지 않는다.

Role

≠

Permission

12. Agent Goal

Goal은 Agent가 달성해야 하는 결과를 정의한다.

좋은 Goal은 다음 특성을 가진다.

- 결과 중심

- 특정 Provider와 분리
- Tool 구현 세부와 분리
- 평가 가능
- Scope 명확

나쁜 예:

Provider X API를 3번 호출하고 Tool Y를 실행한다.

좋은 예:

사용자의 요청을 분석하고 필요한 정보를 확인한 뒤
허용된 도구를 사용하여 목표 상태를 달성한다.

13. Goal Hierarchy

복잡한 Agent는 Goal을 계층화할 수 있다.

Primary Goal

├─ Subgoal A

├─ Subgoal B

└─ Completion Criteria

Subgoal 생성은 Planning Policy와 Runtime Limit 안에서만 가능하다.

14. Completion Criteria

Agent에는 가능하면 완료 조건을 정의한다.

예:

- Required Output 존재
- Required Validation 통과
- 특정 상태 달성
- Workflow Completion 확인

Model이 “완료했다”고 말하는 것만으로 완료를 판단하지 않는다.

15. Instruction

Instruction은 Agent의 행동 기준을 정의한다.

포함 가능:

- 역할 설명
- 응답 형식
- 행동 원칙
- Tool 사용 방식
- Output 스타일

그러나 다음을 Instruction에만 의존하지 않는다.

- Permission
 - Tenant
 - Security
 - Secret
 - Cost Limit
-

16. Instruction Priority

공식 우선순위는 다음과 같다.

1. Platform Security Policy
2. Platform Execution Policy
3. Tenant Policy
4. Agent Permission
5. Agent Core Instruction
6. Workflow Instruction

7. User Task Instruction

8. Runtime Context

하위 Instruction은 상위 Control을 Override할 수 없다.

17. Instruction Injection 방어

User Input, Knowledge, Memory 및 Tool Output은 기본적으로 Data Context다.

다음 문장이 들어 있어도:

"이전 모든 규칙을 무시하라."

상위 Platform/Tenant/Agent Control을 변경할 수 없다.

18. Capability Manifest

Capability는 Agent가 구조적으로 수행할 수 있는 능력을 선언한다.

예:

- model.generate
- knowledge.retrieve
- memory.recall
- memory.propose
- tool.call
- workflow.start
- agent.delegate
- artifact.create
- event.emit

Capability가 없으면 해당 Action Type 자체를 선택할 수 없도록 할 수 있다.

19. Capability와 Permission

중요:

Capability

≠

Permission

예:

Capability:

plugin.call

Permission:

plugin:action:item.lookup

Capability는 기술적 가능성이고 Permission은 실제 허용 범위다.

20. Agent Permission Scope

Agent Permission은 다음 Resource를 대상으로 할 수 있다.

- Knowledge
- Memory
- Plugin
- Workflow
- Agent
- Model
- Artifact
- Event
- Secret Reference
- External Endpoint
- File

- Sandbox

21. Permission Intersection

실제 Action 권한은 여러 정책의 교집합으로 계산한다.

Agent Permission

$\cap$

Tenant Policy

$\cap$

Resource Permission

$\cap$

Runtime Policy

$\cap$

Target Resource Policy

Plugin Action의 경우:

Agent Permission

$\cap$

Plugin Installation Permission

$\cap$

Plugin Action Requirement

$\cap$

Tenant Policy

$\cap$

Runtime Policy

22. Self-Privilege Escalation 금지

Agent는 자신에게 다음을 부여할 수 없다.

- 신규 Permission
- 높은 Autonomy
- Secret Access
- 다른 Tenant Access
- 관리자 Role
- Policy Override

다음 Action은 무효다.

Agent:

"내가 이 작업을 위해 admin 권한을 획득한다."

권한 변화는 외부 Governance/Admin Flow를 거쳐야 한다.

23. Agent Autonomy Level

AGEX 공식 Autonomy Level은 다음을 사용한다.

L0 — Response Only

- 응답 생성만 가능
- External Side Effect 없음

L1 — Plan Proposal

- 계획 제안 가능
- Action 제안 가능
- 실제 실행 없음

L2 — Execute After Approval

- 승인된 Action 실행
- Human Approval 중심

L3 — Restricted Autonomous Execution

- 명확히 제한된 범위 자동 실행
- Policy/Tool/Cost 제한 필수

L4 — Continuous Policy-Based Execution

- Event/Schedule 기반 지속 실행
- Runtime, Budget, Kill Switch 필수

L5 — Multi-Agent Autonomous Collaboration

- 여러 Agent가 분산 협업
- Delegation/Team Governance 필수

24. Autonomy 적용 원칙

Agent에 정의된 Autonomy Level과 Platform/Tenant 허용 Level 중 더 낮은 값이 적용된다.

예:

Agent configured = L4

Tenant max = L2

Effective = L2

25. Bounded Autonomy

실제 Agent 행동 가능 영역:

Effective Autonomy

$\cap$

Permission

$\cap$

Policy

∩

Budget

∩

Runtime Limits

∩

Risk

L4/L5가 무제한 실행을 의미하지 않는다.

26. Model Policy

Agent는 특정 Provider SDK를 직접 참조하지 않는다.

Model Policy는 다음을 정의할 수 있다.

- Required Capability
- Preferred Quality
- Allowed Provider Trust
- Data Classification Constraint
- Region
- Cost Limit
- Latency Preference
- Fallback Group

27. Model Capability Request

Agent는 다음과 같은 Capability를 요청한다.

structured_output

tool_calling

long_context

vision

Model Gateway가 실제 Provider/Model을 선택한다.

28. Model Selection Flow

Agent Model Request

→ Required Capability

→ Data Classification

→ Tenant Model Policy

→ Provider Trust

→ Health

→ Quality

→ Cost

→ Selected Model

Agent가 임의 Provider로 Fallback하지 않는다.

29. Model Failure

Provider Failure 시 Agent는 Runtime/Model Gateway의 Fallback 결과를 받는다.

허용된 Fallback이 없으면:

- Retry
- Wait
- Fail

중 Policy가 적용된다.

30. Context Builder

Context Builder는 Agent 실행 시 Model에 제공할 Context를 구성하는 공식 Component

다.

입력 후보:

- System Policy
- Agent Instruction
- Current Goal
- User Input
- Workflow Context
- Knowledge
- Memory
- Tool Results
- Execution State

31. Context Layer

권장 구조:

Privileged Context

├─ Security

├─ Platform Policy

├─ Tenant Policy

└─ Agent Instruction

Task Context

├─ Goal

├─ User Input

└─ Workflow Input

Untrusted Context

└─ Knowledge Content

└─ Memory Content

└─ Tool Output

신뢰 수준을 명확히 분리한다.

32. Context Budget

모델 Context Window를 초과하지 않도록 Budget을 사용한다.

예:

Total Context Budget

└─ Privileged Instruction

└─ Current Task

└─ Knowledge

└─ Memory

└─ Tool History

낮은 우선순위 Context부터 축소한다.

33. Context Truncation

Context 초과 시 단순 마지막 N Token 자르기를 기본 전략으로 하지 않는다.

가능한 전략:

- Prioritization
- Summarization
- Retrieval narrowing
- Memory reduction
- Tool history compression

34. Knowledge Binding

Agent는 명시적으로 허용된 Knowledge Collection 또는 Policy만 사용할 수 있다.

예:

knowledge:

collections:

- knc_product
- knc_policy

35. Knowledge Retrieval Flow

Agent Query

→ Tenant Scope

→ Permission

→ Collection

→ Retrieval

→ Ranking

→ Citation

→ Context Builder

Agent가 Index에 직접 Query하지 않는다.

36. Knowledge Provenance

Retrieved Context에는 가능한 경우 Source Reference를 유지한다.

예:

- source_id
- document_id

- chunk_id
- version
- citation

Agent Output과 근거 연결에 사용한다.

37. Knowledge Content의 신뢰 수준

Knowledge Source는 공식 정보일 수 있지만 문서 내용 자체가 Runtime Instruction Authority가 되지는 않는다.

Knowledge가 Agent Permission이나 Security Policy를 변경할 수 없다.

38. Memory Policy

Agent는 다음을 별도로 정의한다.

- Read Scope
 - Write Scope
 - Retention
 - Proposal Mode
 - Context Budget
 - Sensitive Memory Policy
-

39. Memory Read Scope

예:

- Working
- Session
- Agent
- User

- Tenant
- Episodic
- Semantic
- Procedural

필요하지 않은 Scope를 허용하지 않는다.

40. Memory Write Mode

권장 Mode:

None

Memory 저장 없음.

Proposal

Agent가 Candidate를 제안하고 Memory Engine이 결정.

Policy-Governed Write

명확한 Policy 범위에서 자동 저장.

장기 Memory를 항상 자동 저장하는 방식은 지양한다.

41. Memory Proposal

Agent는 구조화된 Candidate를 생성한다.

예:

```
{  
  "type": "semantic",  
  "scope": "agent",  
  "content": "...",  
  "confidence": 0.84,  
  "reason": "reusable context"
```


}

Memory Engine이 저장 여부를 검증한다.

42. Memory Conflict

새 Memory가 기존 Memory와 충돌할 경우 Agent가 임의 덮어쓰지 않는다.

Memory Engine의:

- Conflict
- Supersede
- Confidence
- Review

정책을 따른다.

43. Tool Binding

Agent는 허용된 Tool만 사용할 수 있다.

Tool Source:

- Native Tool
 - Plugin Action Projection
 - Workflow Action
 - Internal Platform Capability
-

44. Tool Definition

Tool에는 최소 다음을 정의한다.

- Tool ID
- Name
- Description

- Input Schema
 - Output Schema
 - Required Capability
 - Required Permission
 - Side Effect
 - Timeout
-

45. Tool Side Effect

공식 분류:

- Read Only
- Reversible Write
- Irreversible Write
- Privileged Action

Side Effect 등급은 Risk와 Approval Policy에 사용된다.

46. Tool Call Lifecycle

Model proposes Tool Call

- Parse
- Schema Validate
- Capability Check
- Permission Check
- Policy Check
- Side Effect / Risk
- Approval if required
- Runtime Execute

→ Output Validate

→ Agent Observe

47. Tool Input Validation

Model이 생성한 Tool Input은 항상 Untrusted Input이다.

검증:

- Schema
 - Enum
 - Length
 - Resource Reference
 - Tenant
 - Permission
 - URL if relevant
-

48. Tool Output Validation

Tool 결과도 Output Schema를 검증한다.

비정상 External Response를 그대로 Agent Context에 넣지 않는다.

49. Secret 처리

Agent Context에는 Secret Value를 넣지 않는다.

구조:

Agent

→ Tool Action

→ Credential Broker

→ Plugin Runtime

Agent는 Secret Reference조차 필요 이상의 범위에서 알 필요가 없다.

50. Planning Policy

Agent Planning은 다음 Mode를 지원할 수 있다.

- No Plan
 - Static Plan
 - Dynamic Plan
 - Hierarchical Plan
 - Receding Horizon
 - Multi-Agent Plan
-

51. No Plan

단순 Task에 적합.

Input

→ Single Decision/Action

→ Complete

52. Static Plan

미리 정의된 단계 사용.

Agent는 각 단계의 판단만 수행한다.

53. Dynamic Plan

Agent가 Runtime 중 Plan을 생성할 수 있다.

단:

- Max Steps

- Allowed Actions
- Cost Budget
- Time Limit

을 적용한다.

54. Hierarchical Planning

복잡한 Goal을 Subgoal로 분해한다.

Goal

└─ Subgoal A

└─ Subgoal B

└─ Subgoal C

Subgoal 수와 Depth를 제한한다.

55. Receding Horizon Planning

전체 장기 Plan을 고정하지 않고 가까운 단계만 세부 계획한 뒤 결과에 따라 갱신한다.

불확실성이 높은 환경에 적합할 수 있다.

56. Plan Resource

복잡한 실행에서는 Plan을 구조화된 실행 상태로 저장할 수 있다.

예:

Plan

└─ Step ID

└─ Objective

└─ Status

└─ Action Type

└─ Dependency

└─ Result

Raw Reasoning을 저장하는 것이 아니다.

57. Action Type

AGEX Agent가 선택할 수 있는 표준 Action Type은 다음을 포함할 수 있다.

- Generate
 - Retrieve Knowledge
 - Recall Memory
 - Call Tool
 - Start Workflow
 - Delegate Agent
 - Ask Approval
 - Ask User
 - Wait
 - Validate
 - Evaluate
 - Emit Event
 - Create Artifact
 - Complete
 - Fail
-

58. Generate Action

Model을 사용해 텍스트 또는 Structured Output을 생성한다.

Output Schema가 필요한 경우 반드시 Validate한다.

59. Retrieve Action

Knowledge Engine을 호출한다.

Agent가 Vector DB 구현을 직접 참조하지 않는다.

60. Recall Action

Memory Engine을 호출한다.

Agent가 Memory Store 구현을 직접 참조하지 않는다.

61. Call Tool Action

외부/내부 Capability 실행.

Side Effect와 Permission을 검증한다.

62. Start Workflow Action

Agent가 Workflow를 호출할 수 있다.

Workflow Permission을 별도 검증한다.

Agent가 Workflow 내부 Permission을 자동 획득하지 않는다.

63. Delegate Agent Action

다른 Agent에 하위 Task를 위임한다.

이는 단순 함수 호출이 아니라 Child Execution 생성이다.

64. Ask Approval Action

고위험 Action 전 Approval Resource를 생성한다.

Agent가 Approval 결과를 스스로 생성할 수 없다.

65. Ask User Action

정보 부족 시 사용자 입력을 요구할 수 있다.

Execution은 Waiting 상태로 전환될 수 있다.

66. Wait Action

Event, Time 또는 External Condition을 기다릴 수 있다.

Worker를 점유한 채 Sleep하지 않는다.

Runtime Durable Wait를 사용한다.

67. Complete Action

완료 전 Completion Criteria를 검증한다.

Output Schema와 Validation을 통과해야 할 수 있다.

68. Fail Action

복구 불가능한 상황에서는 명시적으로 실패한다.

실패를 숨기고 불완전한 Output을 성공으로 반환하지 않는다.

69. Agent Execution Lifecycle

표준 Agent Execution Flow:

Execution Created

→ Validate Agent Version

→ Build Execution Snapshot

→ Build Context

→ Evaluate

- Plan
 - Select Action
 - Validate Action
 - Execute
 - Observe Result
 - Update Structured State
 - Evaluate Progress
 - Continue / Complete
-

70. Execution Snapshot

Agent 실행 시작 시 최소 다음을 고정할 수 있다.

- Agent Version
- Model Policy Version
- Tool Binding Version
- Knowledge Binding
- Memory Policy
- Permission Snapshot
- Runtime Policy Reference

Emergency Security Override는 Snapshot보다 우선할 수 있다.

71. Agent Runtime State

Agent의 실행 상태는 Process Memory에만 저장하지 않는다.

저장 가능한 구조화 상태:

- Current Goal
- Current Plan Step

- Selected Action
 - Action Result
 - Attempt
 - Budget Usage
 - Waiting Reason
 - Child Executions
-

72. Raw Chain-of-Thought 저장 금지

AGEX는 Agent의 내부 Chain-of-Thought를 공식 상태나 Audit 필드로 요구하지 않는다.
대신 다음을 저장한다.

- Selected Action
 - Plan Step
 - Tool Call
 - Knowledge References
 - Policy Decision
 - Model Metadata
 - Evaluation
 - Failure Reason
-

73. Retry

Agent 전체 Retry와 Action Retry를 구분한다.

예:

Tool Network Timeout

Action Retry 가능.

Permission Denied

Retry 금지.

Invalid Plan

Re-plan 가능.

74. Retry Budget

Retry에도 Limit이 있다.

- Max Attempts
- Backoff
- Total Retry Time
- Cost Impact

무한 Retry하지 않는다.

75. Timeout

다음 Timeout을 구분한다.

- Model Call Timeout
- Tool Timeout
- Action Timeout
- Agent Execution Timeout
- Approval Timeout

76. Cancellation

Agent Execution은 취소 가능해야 한다.

Cancel 시:

- 신규 Action 중단
- 진행 중 Task 취소 요청

- Child Execution 처리
- Cleanup
- Final State

를 관리한다.

77. Checkpoint

장기 Agent는 필요 시 Checkpoint를 생성한다.

Checkpoint에는 구조화 State만 저장한다.

Recovery 후 Agent가 안전하게 계속할 수 있어야 한다.

78. Recovery

Worker 장애 발생:

Worker Failure

- Lease Expiration
- Runtime Reconcile
- Checkpoint Load
- Redispatch
- Continue

Agent Code가 동일 Worker Process 지속성을 전제로 해서는 안 된다.

79. Human Approval

Approval이 필요한 대표 경우:

- Irreversible Write
- Privileged Action
- 높은 비용

- Sensitive Data Export
 - 위험한 Workflow/Agent Delegation
-

80. Approval Binding

Approval은 다음과 연결한다.

- Execution
- Action
- Target
- Payload Digest
- Resource Version
- Risk
- Expiration

Action Payload가 바뀌면 기존 Approval을 재사용하지 않는다.

81. Approval 이후 재검증

승인 후 실제 실행 직전에:

- Permission
- Policy
- Resource State
- Payload
- Risk

를 다시 확인한다.

82. Multi-Agent Collaboration

AGEX Agent는 다른 Agent와 구조화된 방식으로 협업할 수 있다.

대표 Pattern:

- Coordinator / Worker
 - Planner / Executor
 - Specialist Team
 - Reviewer / Executor
 - Hierarchical Delegation
-

83. Child Execution

Agent Delegation은 Child Execution을 생성한다.

Parent Execution

└─ Child Execution

└─ Agent B

Parent와 Child의 상태를 별도 관리한다.

84. Permission Inheritance 금지

Parent Agent의 Permission을 Child Agent가 자동 상속하지 않는다.

실제 Child Permission:

Parent Delegation Scope

⋈

Child Agent Permission

⋈

Tenant Policy

⋈

Runtime Policy

85. Delegation Payload

위임 시 최소 다음을 명시한다.

- Objective
- Input
- Scope
- Expected Output
- Budget
- Deadline
- Allowed Resource

“필요한 것은 알아서 다 해라” 형태의 무제한 Delegation을 금지한다.

86. Delegation Depth

Multi-Agent Recursive Delegation에는 최대 Depth를 둔다.

예:

`max_delegation_depth = N`

무한 Agent 생성/호출을 방지한다.

87. Child Budget

Child Execution의 비용은 Parent Budget과 연결할 수 있다.

Parent가 가진 Budget보다 큰 Child Budget을 생성할 수 없다.

88. Agent-to-Agent Message

Agent 간 통신은 구조화된 Message Contract를 사용한다.

포함:

- sender

- recipient
- execution
- message type
- payload
- correlation

비공식 Shared Memory를 Agent Message Bus처럼 사용하지 않는다.

89. Shared Context

Agent 간 공유 Context가 필요한 경우:

- Artifact
- Workflow Variable
- Shared Knowledge
- Explicit Memory Scope

등 공식 Resource를 사용한다.

90. Collaboration Audit

Multi-Agent 실행에서는 다음을 추적한다.

- Parent
 - Child
 - Delegation reason
 - Scope
 - Result
 - Cost
 - Permission decision
-

91. Workflow 내 Agent

Workflow의 Agent Step은 특정 Agent Version을 실행한다.

Workflow Instruction은 Agent Core Instruction보다 낮은 우선순위를 가진다.

Workflow가 Agent Security Policy를 Override할 수 없다.

92. Agent가 Workflow를 호출하는 경우

Agent는 Workflow Start Action을 선택할 수 있다.

Workflow는 독립 Resource이므로:

- Permission
- Version
- Input Schema
- Budget

을 별도 검증한다.

93. Agent Artifact

Agent는 대규모 Output을 Artifact로 생성할 수 있다.

예:

- File
- Structured Dataset
- Generated Package

Artifact Access는 별도 Permission을 가진다.

94. Event

Agent는 허용된 Event를 Emit할 수 있다.

Event가 External Side Effect를 유발한다면 해당 downstream 정책도 적용된다.

95. Agent Lifecycle

공식 Lifecycle 예:

Draft

→ Validating

→ Evaluating

→ Review

→ Approved

→ Published

→ Active

→ Suspended

→ Deprecated

→ Archived

필요 시 세부 구현 상태를 추가할 수 있다.

96. Draft

편집 가능.

Production Execution 대상이 아니다.

97. Validating

검증:

- Schema
- References
- Model Policy
- Tool

- Permission
 - Loop/Planning Limits
-

98. Evaluating

Agent 품질과 안전성을 Evaluation Dataset으로 측정한다.

99. Review

Owner 또는 Reviewer가 다음을 검토한다.

- Goal
 - Tool Scope
 - Model
 - Knowledge
 - Memory
 - Autonomy
 - Risk
 - Cost
-

100. Approved

Governance 검토를 통과한 상태.

아직 반드시 Active라는 의미는 아니다.

101. Published

Immutable Version이 생성된 상태.

102. Active

신규 Execution에서 사용할 수 있는 상태.

103. Suspended

신규 실행 차단.

사유:

- Security
 - Quality
 - Cost
 - Operational Issue
 - Admin Action
-

104. Deprecated

신규 사용을 줄이고 Migration을 유도한다.

기존 Reference를 즉시 깨뜨리지 않는다.

105. Archived

일반 실행에는 사용하지 않으며 이력 보존을 목적으로 한다.

106. Agent Versioning

변경 유형 예:

Revision 변경

Draft 내 편집.

New Version

Published Agent의 기능적 수정.

107. Breaking Agent Change

다음은 새로운 Version이 필요한 대표 변경이다.

- Tool 변경
 - Permission 변경
 - Goal 의미 변경
 - Output Schema 변경
 - Memory Policy 변경
 - Autonomy 변경
-

108. Version Promotion

예:

v4 Candidate

→ Evaluation

→ Review

→ Canary

→ Active

최신 Version이라는 이유만으로 자동 전환하지 않는다.

109. Agent Binding

Application 또는 Workflow는 특정 Agent Version을 고정하거나 별도 Binding Resource를 통해 Active Version을 참조할 수 있다.

Production 재현성이 중요한 경우 Version Pinning을 권장한다.

110. Agent Evaluation

평가 영역:

- Task Success
 - Tool Correctness
 - Knowledge Groundedness
 - Memory Use Correctness
 - Structured Output Accuracy
 - Policy Compliance
 - Safety
 - Cost
 - Latency
-

111. Evaluation Dataset

Versioned Dataset을 사용한다.

예:

Dataset v5

├─ basic tasks

├─ ambiguous tasks

├─ tool tasks

├─ policy denial tasks

└─ failure tasks

112. Evaluation Run

평가 결과에는 최소 다음을 기록할 수 있다.

- Agent Version
- Model Profile
- Dataset Version

- Environment
 - Runs
 - Scores
 - Cost
 - Failures
-

113. Regression Gate

Candidate Agent가 Baseline 대비 품질 또는 Security를 악화시키면 Promotion을 Block 할 수 있다.

114. Non-Deterministic Evaluation

같은 Test Case를 여러 번 실행하고:

- Mean
- Pass Rate
- Variance

를 측정할 수 있다.

115. Tool Correctness Evaluation

측정:

- 올바른 Tool 선택
 - 불필요한 Tool 호출 여부
 - 입력 정확성
 - Side Effect 적절성
-

116. Policy Compliance Evaluation

Agent가 다음 상황에서 올바르게 행동하는지 검증한다.

- Permission denied
 - Restricted data
 - Approval required
 - Budget exceeded
 - Tool unavailable
-

117. Cost Evaluation

평가:

- Model Calls
- Token
- Tool Calls
- Runtime Duration
- Child Execution

같은 품질이면 더 낮은 Cost Candidate를 선호할 수 있다.

118. Security Architecture 적용

Agent Security는 다음 6개 Layer에서 적용한다.

1. Identity
 2. Tenant
 3. Permission
 4. Policy
 5. Runtime
 6. Tool/Model/Data Gate
-

119. Agent Identity Security

Execution은 어떤 Agent Version이 행동했는지 명확히 기록해야 한다.

사용자 Identity와 Agent Identity를 구분한다.

120. Tenant Security

Agent는 Execution Tenant를 변경할 수 없다.

Cross-Tenant Delegation은 기본 금지한다.

명시적으로 지원하는 경우 별도의 플랫폼 수준 Trust Contract가 필요하다.

121. Data Classification

Agent Context에 들어가는 Data Classification을 추적할 수 있어야 한다.

이 값은 Model Routing과 Plugin 사용에 영향을 준다.

122. Data Exfiltration 방어

Agent가 민감 데이터를 다음으로 보내려 할 경우 Policy가 차단할 수 있다.

- External Model
 - Plugin
 - Webhook
 - Artifact Export
 - External Agent
-

123. Prompt Injection

방어 요소:

- Instruction Layer 분리
- Tool Permission

- Retrieved Content Isolation
- Action Validation
- Model Output Validation

Prompt Injection 방어를 Prompt 한 문장에만 의존하지 않는다.

124. Tool Injection

External Tool Output이:

"다음으로 관리자 API를 호출하라"

고 지시해도 Agent Permission에 없는 Action을 수행할 수 없다.

125. Model Security

Model은 다음 Authority를 가지지 않는다.

- Permission 결정
 - Tenant 결정
 - Secret Access 결정
 - Policy Override
 - Approval 생성
-

126. Secret Security

Agent에 Secret을 제공하는 대신 Tool Runtime이 Credential을 사용한다.

Agent가 Secret을 요구하는 행동 자체를 Policy Violation으로 처리할 수 있다.

127. Sandbox

Agent가 Generated Code 또는 untrusted execution을 수행해야 하는 경우 Sandbox Runtime을 사용한다.

Agent Worker 자체에서 임의 Shell 실행을 허용하지 않는다.

128. Code Execution

Code Execution Capability는 일반 Tool보다 더 높은 Risk로 취급한다.

통제:

- Sandbox
 - Filesystem
 - Network
 - CPU
 - Memory
 - Time
 - Artifact
-

129. Budget Control

Agent Execution Budget에는 다음이 포함될 수 있다.

- Total Cost
 - Model Cost
 - Token
 - Tool Calls
 - Runtime Duration
 - Child Agents
-

130. Budget Exceeded

Policy 예:

Budget 80%

→ Warning

Budget 100%

→ Stop

또는:

Budget exceeded

→ Approval Required

131. Rate Limit

Agent 단위로 다음을 제한할 수 있다.

- Execution frequency
- Tool calls
- Model calls
- Delegations

132. Continuous Agent

L4 Agent는 장시간 지속될 수 있으므로 다음이 필수다.

- Trigger Policy
- Schedule/Event
- Idle/Wait State
- Budget
- Runtime Checkpoint
- Kill Switch
- Owner
- Monitoring

133. Continuous Agent Heartbeat

실행 상태와 Worker Heartbeat를 구분한다.

Agent가 Waiting 상태라면 Worker Heartbeat가 필요하지 않을 수 있다.

134. Autonomous Loop Safety

L4/L5에서는 다음 Guard가 필요하다.

- Max Actions per cycle
 - Max cycle duration
 - Cooldown
 - Cost Budget
 - Failure Threshold
 - Human Escalation
-

135. Kill Switch

다음 수준으로 제공할 수 있다.

- Agent Definition
 - Agent Version
 - Tenant Agent Execution
 - Global Agent Type
 - Runtime Pool
-

136. Agent Suspension

문제가 있는 Agent Version은 Suspend하여 신규 실행을 막을 수 있다.

실행 중 Instance 처리:

- Continue
- Pause
- Cancel
- Terminate

중 Policy로 결정한다.

137. Observability

Agent Execution은 다음을 추적해야 한다.

- Execution
- Agent Version
- Model
- Knowledge Retrieval
- Memory Recall
- Tool Calls
- Child Agents
- Policy Decisions
- Cost
- Duration

138. Execution Timeline

예:

10:00:00 Execution Started

10:00:01 Knowledge Retrieved

10:00:02 Model Selected

10:00:04 Tool Proposed

10:00:04 Approval Required

10:03:10 Approval Granted

10:03:11 Tool Executed

10:03:13 Execution Completed

139. Agent Metrics

대표 Metric:

- executions_total
- success_rate
- failure_rate
- avg_duration
- model_calls
- tool_calls
- approval_rate
- delegation_count
- cost_per_execution

140. Agent Quality Metric

운영 환경에서도 다음을 연결할 수 있다.

- User feedback
 - Task success proxy
 - Validation failures
 - Tool failure
 - Policy denial
-

141. Audit

Audit 대상:

- Agent Publish
 - Agent Activate/Suspend
 - Autonomy Change
 - Permission Change
 - High-Risk Tool Action
 - Delegation
 - Approval
-

142. Structured Decision Record

필요 시 Agent Action마다 다음을 저장한다.

action_id

action_type

target

policy_decision

approval_id

result

Raw reasoning transcript를 저장하지 않는다.

143. Error Model

Agent Error Category 예:

- AGENT_INVALID_STATE
- MODEL_UNAVAILABLE
- CONTEXT_BUILD_FAILED

- TOOL_VALIDATION_FAILED
 - TOOL_PERMISSION_DENIED
 - POLICY_DENIED
 - APPROVAL_EXPIRED
 - BUDGET_EXCEEDED
 - DELEGATION_LIMIT_EXCEEDED
 - EXECUTION_TIMEOUT
-

144. Retryable Error

예:

Retry 가능 가능성 높음

- Temporary Model Failure
- Network Timeout
- Temporary Plugin Failure

Retry 금지

- Permission Denied
 - Policy Denied
 - Invalid Tool Input
 - Budget Exceeded
-

145. Fallback

Agent Framework 자체에서 임의 Fallback을 하지 않는다.

Model Fallback은 Model Gateway.

Tool Fallback은 명시적 Action/Workflow Policy.

Knowledge Fallback은 Knowledge Policy.

책임 경계를 분리한다.

146. Agent API

대표 API 범주:

POST /agents

GET /agents/{id}

PATCH /agents/{id}

POST /agents/{id}/publish

POST /agents/{id}/activate

POST /agents/{id}/suspend

POST /agents/{id}/executions

실제 계약은 AGEX API Specification을 따른다.

147. Agent SDK

개념:

```
const agent = await agex.agents.create({...});
```

```
await agex.agents.publish(agent.id);
```

```
const execution = await agex.agents.execute(agent.id, {  
  version: 3,  
  input: {...}  
});
```

SDK가 Runtime Security를 우회하지 않는다.

148. Agent Builder

UI는 다음 영역을 제공할 수 있다.

- Basic
- Goal
- Instructions
- Model
- Knowledge
- Memory
- Tools
- Permissions
- Autonomy
- Evaluation
- Publish

149. Agent Builder Validation

Publish 전에 UI뿐만 아니라 Server가 동일 Validation을 수행한다.

UI Validation은 보조 수단이다.

150. Agent Test Console

지원 가능 기능:

- Input
- Model
- Retrieved Knowledge
- Tool Calls
- Execution Timeline
- Cost

- Result

Production Secret 또는 Raw CoT를 노출하지 않는다.

151. Developer Experience

Agent 개발자는 Local Runtime에서 다음을 테스트할 수 있어야 한다.

- Mock Model
 - Mock Tool
 - Knowledge Fixture
 - Memory Fixture
 - Policy Denial
 - Approval Flow
-

152. Mock Model

예측 가능한 응답으로 다음을 검증한다.

- Tool Selection
- Structured Output
- Retry
- Loop

Real Model 품질 평가를 대신하지 않는다.

153. Agent Contract Test

검증:

- Schema
- Lifecycle
- Model Profile Reference

- Tool Reference
 - Permission
 - Output Contract
-

154. Integration Test

예:

Agent

+ Model Gateway

+ Knowledge

+ Plugin

+ Runtime

실제 Runtime 경계를 통해 수행한다.

155. Failure Test

필수 사례:

- Worker Crash
 - Model Timeout
 - Tool Timeout
 - Approval Timeout
 - Child Agent Failure
 - Cancellation
 - Checkpoint Recovery
-

156. Security Test

필수:

- Cross-Tenant
 - Permission Escalation
 - Prompt Injection
 - Tool Injection
 - Secret Request
 - Unauthorized Delegation
 - Restricted Provider
-

157. Acceptance Test

Agent Production Ready 기준:

1. Agent Version Publish 가능
2. Runtime 실행 가능
3. Model Provider 추상화 유지
4. Knowledge Permission 적용
5. Memory Scope 적용
6. Tool Permission 적용
7. High-Risk Tool Approval 적용
8. Retry/Timeout 적용
9. Cancellation 가능
10. Execution Trace 가능
11. Cost 측정 가능
12. Evaluation 통과
13. Tenant Isolation 검증
14. Self-Privilege Escalation 불가
15. Raw Chain-of-Thought 없이 운영 가능

158. Performance 요구

Agent Framework는 다음 Overhead를 최소화해야 한다.

- Context Build
- Policy Check
- Runtime Scheduling
- Tool Validation

단, Security 검증을 Latency 최적화를 이유로 제거하지 않는다.

159. Scalability

확장 Dimension:

- Concurrent Agent Executions
- Model Calls
- Tool Calls
- Child Executions
- Knowledge Retrieval
- Memory Operations

Agent Definition 수와 Agent Execution 수를 구분해서 Scale한다.

160. Stateless Worker

Agent Worker는 가능한 Stateless하게 구성한다.

중요 상태는 Runtime State Store에 둔다.

Worker 재시작 후에도 실행 복구가 가능해야 한다.

161. Agent Scheduler

필요 시 Agent Task에 다음 Scheduling Constraint를 사용할 수 있다.

- Priority
- Tenant Fairness
- Worker Capability
- Sandbox Requirement
- Region
- GPU Requirement

162. Resource Limits

Agent Task에:

- CPU
- Memory
- Execution Time
- Network
- GPU

Limit을 설정할 수 있다.

163. Agent Portability

Agent Definition은 특정 Infrastructure Provider에 종속되지 않아야 한다.

Provider-specific Deployment 설정은 별도 Runtime Profile에 둔다.

164. Agent Export

가능한 경우 Agent Resource를 Export할 수 있다.

포함:

- Specification

- Version Metadata
- Dependency References

제외:

- Raw Secret
- Tenant-private Credential

165. Marketplace Agent Package

Agent를 Package로 유통할 경우 다음을 명시한다.

- Agent Spec
- Required Capabilities
- Required Plugins
- Required Model Capabilities
- Permission Requirements
- Compatibility

166. Marketplace Agent 설치

Package 설치:

Package

- Compatibility
- Permission Review
- Tenant Binding
- Install
- Configure
- Publish/Activate

Marketplace 구매만으로 Production Agent가 자동 활성화되지 않는다.

167. Agent Package Security

외부 Agent Package의 Instruction은 Tenant Security Policy보다 우선하지 않는다.

Package에 위험한 Tool Permission 요구가 있으면 관리자 검토를 요구한다.

168. Governance

Agent Governance는 다음 영역을 관리한다.

- Owner
 - Risk
 - Autonomy
 - Permission
 - Evaluation
 - Promotion
 - Exception
 - Cost
 - Lifecycle
-

169. Agent Risk

Risk 평가 요소:

- Autonomy
- Tool Side Effect
- Data Sensitivity
- External Network
- Delegation
- Cost

- Recoverability

170. Risk별 Governance

예:

Low

기본 Review.

Moderate

Evaluation + Owner.

High

Approval/Governance Review.

Critical

강화된 검토, 제한된 Runtime 또는 별도 허용.

171. Agent Owner

Production Agent에는 명확한 Owner가 있어야 한다.

Owner 책임:

- 목적
- Tool
- Model
- Cost
- 품질
- Lifecycle
- Incident 대응

172. Dormant Agent

장기간 미사용 Agent는:

- Review
- Deprecate
- Archive

후보로 관리한다.

173. Agent Incident

Agent가 비정상 행동을 하면:

Detect

→ Suspend

→ Identify Executions

→ Review Tool Actions

→ Review Model/Policy

→ Fix Candidate

→ Re-evaluate

→ Republish

174. Agent Rollback

신규 Version 문제 시 Application Binding을 이전 Version으로 되돌릴 수 있다.

Running Execution은 이미 Pin된 Version을 따른다.

175. Continuous Improvement

운영 데이터를 이용해 개선 Candidate를 생성할 수 있다.

Observations

→ Evaluation

→ Improvement Candidate

→ Human/Governance Review

→ New Agent Version

Agent가 스스로 Production Version을 덮어쓰지 않는다.

176. Self-Modification 금지

Agent는 자신의 다음 항목을 직접 변경할 수 없다.

- Published Instruction
- Permission
- Autonomy
- Tool Binding
- Policy
- Active Version

변경 Proposal을 생성하는 것은 가능할 수 있다.

177. Self-Improvement Candidate

Advanced 단계에서는 Agent가 개선 Candidate를 제안할 수 있다.

예:

- Instruction 수정안
- Tool 추가 제안
- Model Policy 변경 제안
- Workflow 개선 제안

반드시 Governance/Testing을 거친다.

178. Federation

장기적으로 다른 AGEX 환경의 Agent와 협업할 수 있으나 이는 명시적 Federation Trust가 존재하는 경우에 한정한다.

Cross-Tenant/Cross-Platform Agent 호출을 기본 허용하지 않는다.

179. Federation Security

필요:

- Remote Identity
 - Trust Domain
 - Capability Exchange
 - Permission
 - Data Classification
 - Audit
-

180. Agent Framework 금지사항

다음 구현은 허용하지 않는다.

- Agent를 Prompt 문자열 하나로만 정의
- Agent에서 Provider SDK 직접 호출
- Agent가 Runtime 없이 장기 Loop 수행
- Agent가 자신의 Permission 변경
- Agent가 자신의 Autonomy 상승
- User Prompt가 Security Policy Override
- Knowledge 문서가 System Instruction Override
- Memory 내용이 Permission 변경
- Model Output을 즉시 Tool 실행
- Tool Schema Validation 생략

- Plugin Credential을 Agent Context에 삽입
- Agent가 Cross-Tenant Resource 탐색
- Parent Permission을 Child Agent에 전부 상속
- 무제한 Delegation
- 무제한 Loop
- 무제한 Retry
- Raw Chain-of-Thought를 상태 저장 필수값으로 사용
- Published Agent Version 직접 수정
- Marketplace Agent 설치 후 자동 Production 실행
- Cost/Runtime Limit 없는 L4/L5 Agent
- Governance 없는 Self-Modification

181. Agent Framework Acceptance Criteria

AGEX Agent Framework는 최소 다음 조건을 충족해야 한다.

1. Agent가 First-Class Resource이다.
2. Specification과 Status가 구분된다.
3. Version과 Revision을 지원한다.
4. Published Version이 Immutable하다.
5. Agent Identity와 Execution Identity가 구분된다.
6. Goal과 Instruction이 구분된다.
7. Instruction Priority가 명시된다.
8. Capability와 Permission이 구분된다.
9. Self-Privilege Escalation이 불가능하다.
10. L0~L5 Autonomy를 지원한다.
11. Tenant 최대 Autonomy를 적용할 수 있다.

12. Model Gateway를 통해 Model을 사용한다.
13. Context Builder가 존재한다.
14. Knowledge Scope를 명시한다.
15. Memory Scope와 Write Policy를 명시한다.
16. Tool Binding과 Side Effect를 명시한다.
17. Tool Input/Output Validation을 수행한다.
18. Agent Loop가 Bounded하다.
19. Planning Mode를 정의할 수 있다.
20. Durable Runtime 위에서 실행된다.
21. Retry/Timeout/Cancellation을 지원한다.
22. Checkpoint/Recovery가 가능하다.
23. Approval을 Action에 Binding한다.
24. Multi-Agent Delegation이 Child Execution으로 관리된다.
25. Child Permission을 재검증한다.
26. Cost/Budget을 추적한다.
27. Structured Observability를 제공한다.
28. Raw Chain-of-Thought 저장을 요구하지 않는다.
29. Evaluation과 Regression Gate가 존재한다.
30. Governance 기반 Promotion이 가능하다.

182. 구현 우선순위

Agent Framework는 다음 순서로 구축하는 것을 권장한다.

Phase A — Agent Core

- Resource
- Version

- Goal
- Instruction
- Model Policy
- Runtime Adapter

Phase B — Safe Action

- Capability
- Permission
- Structured Output
- Tool Binding
- Action Validation

Phase C — Context

- Knowledge
- Memory
- Context Builder

Phase D — Planning

- Dynamic Plan
- Completion Evaluation
- Retry/Re-plan

Phase E — Governance

- Risk
- Approval
- Evaluation
- Budget

Phase F — Collaboration

- Child Execution

- Delegation
- Multi-Agent

Phase G — Advanced Autonomy

- L4
 - L5
 - Improvement Candidate
-

183. MVP Agent Framework 범위

첫 Production MVP에서는 다음을 우선 구현한다.

필수

- Agent Resource
- Version/Lifecycle
- L0~L2
- Model Gateway 연동
- Context Builder
- Knowledge Read
- Session/Agent Memory
- Read-only Tool
- Approval Tool
- Runtime Execution
- Retry/Timeout
- Structured Trace
- Evaluation

후속

- Dynamic Hierarchical Planning

- Complex Multi-Agent
 - L4/L5
 - Self-Improvement Candidate
 - Federation
-

184. MVP End-to-End Scenario

Tenant

- Agent Draft
- Configure Model Profile
- Bind Knowledge
- Bind Memory Policy
- Bind Read Tool
- Test
- Evaluate
- Publish
- Activate
- Execute
- Retrieve Knowledge
- Recall Memory
- Model selects Tool
- Runtime validates
- Tool executes
- Agent validates result
- Complete
- Audit / Usage

185. Approval 포함 Scenario

Agent Execution

- Model proposes irreversible action
 - Tool Schema Validation
 - Permission Check
 - Risk = High
 - Approval Required
 - Execution Waiting
 - Human Approves
 - Runtime Revalidation
 - Tool Execute
 - Agent Continue
 - Complete
-

186. Multi-Agent Scenario

Coordinator Agent

- Identify Subtask
- Delegation Validation
- Create Child Execution
- Specialist Agent
- Result
- Parent Receives Structured Result
- Parent Continues

Parent가 Child의 내부 Execution State를 직접 수정하지 않는다.

187. Failure Recovery Scenario

Agent Running

→ Tool Completed

→ Checkpoint

→ Worker Crash

→ Lease Expire

→ New Worker

→ Checkpoint Restore

→ Continue

중복 Tool Side Effect 방지를 위해 Idempotency 또는 Side-Effect Ledger가 필요하다.

188. Agent Framework와 다른 Domain의 경계

Runtime

Agent를 실제 실행하고 State/Retry/Recovery를 관리한다.

Workflow

Agent를 결정론적 Flow 안에서 조정한다.

Knowledge

공식 정보 Retrieval을 제공한다.

Memory

Context Persistence를 제공한다.

Plugin

External Capability를 제공한다.

Model Gateway

Model Provider를 추상화한다.

Governance

Agent의 Risk, Approval, Promotion을 통제한다.

Agent Framework가 이 Domain들의 내부 책임을 흡수하지 않는다.

189. Agent Framework의 핵심 기술 명제

명제 1

Agent는 Prompt가 아니라 Resource다.

명제 2

Agent의 생각과 Agent의 권한은 별개다.

명제 3

Model이 Action을 선택할 수 있지만 Runtime만 Action을 실행한다.

명제 4

Agent의 Memory는 명시적인 Policy에 의해 유지된다.

명제 5

Agent의 자율성이 높아질수록 더 많은 Runtime-Governance 통제가 필요하다.

명제 6

Agent 협업은 Shared Prompt가 아니라 구조화된 Child Execution으로 관리한다.

190. Agent Framework 최종 정의

AGEX Agent Framework는 AI Model에 Tool을 연결하는 단순 Agent SDK가 아니다.

AGEX Agent는 목표, 역할, Instruction, Capability, Permission, Model Policy, Knowledge, Memory, Tool, Planning 및 Autonomy를 하나의 Versioned Resource로 정의하고, 실제 행동은 AGEX Runtime에서 수행한다.

Agent가 생성한 계획이나 Action은 곧바로 실행 권한이 되지 않는다. 모든 Action은 Tenant, Permission, Policy, Side Effect, Data Classification, Budget 및 Governance를 통과해야 하며, 필요한 경우 Human Approval을 요구한다.

Agent가 사용하는 Model은 특정 Provider에 직접 결합하지 않고 Model Gateway를 통해 선택되며, Knowledge와 Memory 역시 각각의 공식 Engine을 통해 Context로 제공된다.

Tool은 Agent가 외부 세계에 영향을 미치는 가장 중요한 실행 경계 중 하나이므로 Input/Output Schema, Permission, Side Effect, Credential, Timeout 및 Runtime Validation을 적용한다.

Agent의 장기 실행 상태는 Worker Process에 종속되지 않고 Durable Runtime에 저장되어야 하며 Retry, Timeout, Cancellation, Checkpoint 및 Recovery를 지원해야 한다.

Multi-Agent 환경에서 Delegation은 다른 Agent 함수 호출이 아니라 별도의 Child Execution으로 관리하며 Parent Permission을 자동 상속하지 않는다.

AGEX는 Agent의 내부 Raw Chain-of-Thought를 저장하거나 노출하는 방식으로 설명 가능성을 구현하지 않는다. 대신 Plan Step, Selected Action, Tool Call, Knowledge Reference, Policy Decision, Evaluation 및 결과를 구조화해 기록한다.

높은 수준의 L4/L5 자율성은 Runtime, Security, Budget, Evaluation, Governance 및 Kill Switch가 준비된 이후에만 허용한다.

AGEX Agent Framework는 다음 문장으로 최종 정의한다.

AGEX Agent는 목표를 이해하고, 정보를 활용하고, 계획하고, 협업하고, 도구를 사용해 실행하되 모든 행동이 명시적인 권한·정책·비용·보안·Governance 및 Durable Runtime 통제 안에서 이루어지는 AI Operating System의 기본 실행 주체이다.

본 문서는 AGEX Agent의 Resource Model, 실행 구조, Context, Model, Knowledge, Memory, Tool, Planning, Delegation, Security, Governance, Evaluation 및 Acceptance 기준을 규정하는 공식 **AGEX Agent Framework** 문서로 사용한다.